

Pedro Henrique Prado de Novaes

Atividade de revisão para a P3 da matéria de Banco de Dados

QUESTÃO 1 — Modelagem (Embedding x Referencing) A equipe deseja armazenar informações dos carros e também histórico de manutenções, contendo: data, tipo, mecânico responsável e peças trocadas.

```
// Coleção: carros

{
  _id: ObjectId("..."),
  nome: "GT-R #12",
  equipe: "ThunderRacing",
  ano: 2024,
  motor: "V6 Biturbo",

  // Histórico de manutenções embutido (embedding)
  manutencoes: [
    {
      data: ISODate("2025-03-10T10:15:00Z"),
      tipo: "Troca de pneus",
      mecanico: "Carlos Souza",
      pecas_trocadas: ["Pneu dianteiro esquerdo", "Pneu dianteiro direito"]
    },
  ],
}
```

QUESTÃO 2 — Inserção de Dados (CRUD / Aula 3) Insira 3 registros de telemetria na coleção leituras contendo: carro, sensor, valor e data/hora.

```
telemetria_race> db.leituras.insertMany([
  {
    carro: "GT-R",
    sensor: "temperatura_motor",
    valor: 95,
    dataHora: ISODate("2025-03-10T10:00:00Z")
  },
  {
    carro: "GT-R",
    sensor: "pressao_oleo",
    valor: 88,
    dataHora: ISODate("2025-03-10T10:00:05Z")
  },
  {
    carro: "F1-Alpha",
    sensor: "velocidade",
    valor: 310,
    dataHora: ISODate("2025-03-10T10:00:10Z")
  }
]);
```

QUESTÃO 3 — Consultas com Operadores Lógicos (Aula 4) Liste todas as leituras onde: • o sensor seja “temperatura_motor” OU “pressao_oleo”, • E o valor seja maior que 90.

```
> db.leituras.find({
  sensor: { $in: ["temperatura_motor", "pressao_oleo"] }, // OU
  valor: { $gt: 90 }                                     // E valor > 90
});
< {
  _id: ObjectId('692909a053a9d346e5fa76ae'),
  carro: 'GT-R',
  sensor: 'temperatura_motor',
  valor: 95,
  dataHora: 2025-11-28T02:32:00.649Z
}
{
  _id: ObjectId('692909ee93da30e566b0d7c7'),
  carro: 'GT-R',
  sensor: 'temperatura_motor',
  valor: 95,
  dataHora: 2025-03-10T10:00:00.000Z
}
telemetria_race>
```

QUESTÃO 4 — Atualização Avançada (Aula 4) O carro “GT-R” teve uma falha no sensor de temperatura. Atualize todas as leituras desse carro adicionando o campo: "status_sensor": "verificar" E remova o campo codigo_defeito, caso exista

```
> db.leituras.updateMany(
  { carro: "GT-R" }, // filtro: todas as leituras do GT-R
  {
    $set: {
      status_sensor: "verificar" // adiciona/atualiza status_sensor
    },
    $unset: {
      codigo_defeito: "" // remove o campo se existir
    }
  }
);
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

QUESTÃO 5 — Paginação (Aula 4) Liste as 5 leituras mais recentes do sensor “velocidade”, ignorando as primeiras 10.

```
>
db.leituras.find(
  { sensor: "velocidade" }
)
.sort({ dataHora: -1 })
.skip(10)
.limit(5);
```

QUESTÃO 6 — Agregação (Aula 6) Calcule a média de temperatura do motor por carro, ordenando os maiores valores primeiro.

```
> db.leituras.aggregate([
  {
    $match: { sensor: "temperatura_motor" }
  },
  {
    $group: {
      _id: "$carro",
      mediaTemperatura: { $avg: "$valor" }
    }
  },
  {
    $sort: { mediaTemperatura: -1 }
  }
]);
< {
  _id: 'GT-R',
  mediaTemperatura: 95
}
telemetria_race>
```

QUESTÃO 7 — API Node.js (Aula 7) A equipe precisa registrar uma nova leitura via API REST. Crie a rota POST /leituras usando Express + Mongoose

The screenshot shows a VS Code editor with two files open: `leituras.js` in the `routes` folder and `server.js` in the `serverjs` folder. The `leituras.js` file contains a POST route that validates input fields and creates a new `Leitura` document in the database. The `server.js` file shows the Express app setup, including middleware for JSON and CORS, database connection with Mongoose, and the routing module.

```
leituras.js
1 const express = require("express");
2 const router = express.Router();
3 const Leitura = require("../models/Leitura");
4
5 router.post("/", async (req, res) => {
6   try {
7     const { carro, sensor, valor, dataHora }
8
9
10
11     if (!carro || !sensor || valor === undef
12       return res.status(400).json({
13         mensagem: "Campos obrigatórios:
14       });
15
16     const novaLeitura = new Leitura({
17       carro,
18       sensor,
19       valor,
20       dataHora
21     });
22
server.js
1 st express = require("express");
2 st mongoose = require("mongoose");
3 st cors = require("cors");
4
5 st leiturasRoutes = require("../routes/leituras")
6
7 st app = express();
8
9 .use(express.json());
10 .use(cors());
11
12 st MONGO_URL = "mongodb://127.0.0.1:27017/teleme
13
14 goose
15 .connect(MONGO_URL)
16 .then(() => {
17   console.log("Conectado ao MongoDB", MONGO_U
18 })
19 .catch((erro) => {
20   console.error("Erro ao conectar ao MongoDB"
21 });
22
```

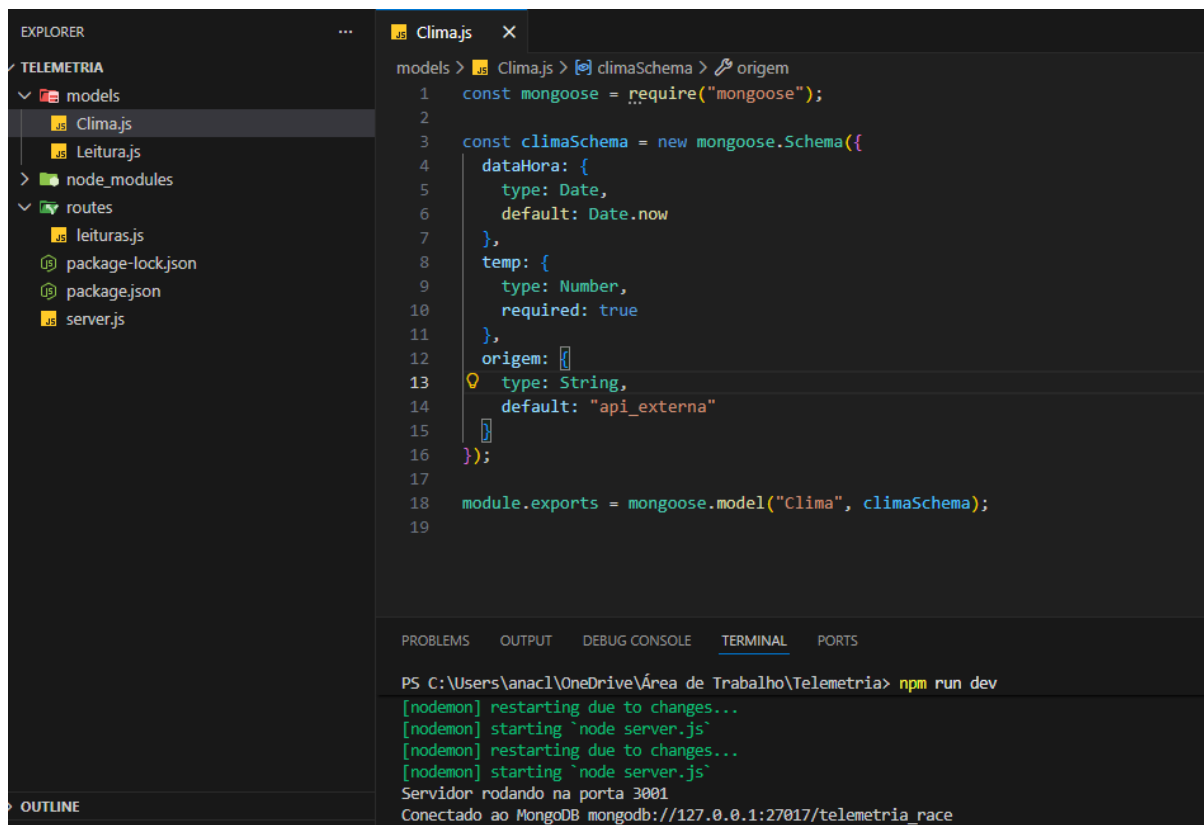
Terminal output:

```
PS C:\Users\anacl\OneDrive\Área de Trabalho\Telemetria> npm run dev
[nodemon] starting 'node server.js'
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting 'node server.js'
Servidor rodando na porta 3001
Conectado ao MongoDB mongodb://127.0.0.1:27017/telemetria_race
```

The screenshot shows a VS Code editor with the `climaService.js` file open in the `services` folder. The file implements an asynchronous function `importarClimaAtual` that fetches weather data from the OpenWeatherMap API, creates a `Clima` document, and saves it to the database. The Explorer sidebar shows the project structure with folders for `models`, `routes`, and `services`.

```
climaService.js
1 require("dotenv").config();
2 const axios = require("axios");
3 const Clima = require("../models/Clima");
4
5 async function importarClimaAtual(cidade = "São Paulo") {
6   try {
7     const apiKey = process.env.API_KEY;
8
9     if (!apiKey) {
10       throw new Error("Chave da API não configurada no arquivo .env");
11     }
12
13     const url = `https://api.openweathermap.org/data/2.5/weather?q=${cidade}&appid=${apiKey}&units=metr
14
15     const resposta = await axios.get(url);
16
17     const temp = resposta.data.main.temp;
18
19     const registroClima = new Clima({
20       temp
21     });
22
23     await registroClima.save();
24
25     console.log("Clima importado com sucesso: ", registroClima);
26
27     return registroClima;
28   } catch (erro) {
29     console.error("Erro ao importar clima: ", erro.message);
30   }
31 }
32 module.exports = { importarClimaAtual };
```

QUESTÃO 8 — Consumo de API Externa (Aula 7) A equipe deseja importar automaticamente a temperatura ambiente antes das corridas.



QUESTÃO 9 — Segurança no MongoDB (Aula 8) Crie um usuário chamado `engenheiroCorrida` com acesso apenas de leitura ao banco `telemetria_race`.

```
> db.createUser({
  user: "engenheiroCorrida",
  pwd: "SenhaForte123", // coloque uma senha adequada
  roles: [
    {
      role: "read", // apenas leitura
      db: "telemetria_race"
    }
  ]
});
< { ok: 1 }
telemetria_race>
```

QUESTÃO 10 — Backup e Restauração (Aula 8) A equipe quer fazer backup diário dos dados de telemetria. Crie o comando de backup e um exemplo de automação com node-cron

